JWT 게시판 구현 보고서

이 양식은 본인이 구현한 결과를 설명하기 위한 문서입니다. 대괄호 안내를 실제 내용으로 바꾸고 작성 안내 문장은 제출 전에 지우세요. 화면 캡처와 표의 행은 필요에 따라 추가할 수 있습니다. 6~10 쪽 내외를 권장하며 분량 자체는 평가하지 않습니다.

01 서비스 소개와 구현 범위

학번 202346113 이름: 송세화 제출일: 2026.10.28

서비스 이름 Mario Secret Guide

기술 조합과 버전 [React / FastAPI / SQLite 선정

GitHub 저장소 URL https://github.com/sehwa-song/mario-secret-guide

소스 코드 기준 main / 8d96614 의 전체 SHA 를 기록할 수 있습니다. 최종 제출 기준 커밋 SHA 는 LMS 에 제출합니다.]

서비스의 대상 사용자와 사용 목적을 3~5 문장으로 설명하세요. 이 기술 조합을 선택한 이유와 제공 템플릿에서 변경한 범위를 적으세요.

본 서비스인 Mario Secret Guide 는 닌텐도 DS 게임인 뉴 슈퍼 마리오브라더스의 숨겨진 월드 및 비밀 출구 공략 정보를 사용자들이 공유할 수 있도록 구현한 웹서비스이다.

게임 플레이 과정에서 발견한 공략 방법을 게시글로 등록하고, 다른 사용자가 이를 자유롭게 조회할 수 있도록 공개 게시판 구조를 채택하였다. 공략 정보의 무단 수정과 삭제를 방지하기 위해 JWT 기반 사용자 인증 및 작성자 권한 검사를 적용하였다.

프론트엔드는 React 와 Vite 를 사용하였으며, 백엔드는 FastAPI, 데이터 저장에는 SQLite 를 활용하였다. 현재 구현 범위는 공략 게시글의 조회·등록·수정·삭제와 사용자 인증 및 권한 관리 기능이다.

상태 관리, 작성자 권한 검사를 구현할 계획이다. (마지막 문장 제출 전에 변경하기)

완료 여부는 완료·부분·미완료 중 하나로 기록하세요.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	완료	main.jsx, api.js, app.py, store.py	로그인 없이 슈퍼마리오 비밀 맵 공략 게시글의 목록과 상세 내용을 조회할 수 있음.
로그인과 JWT 발급	완료	app.py, auth.py, users.py, api.js	Alice 와 Bob 계정의 로그인 및 JWT 발급 구현. 기본 토큰 유효 시간은 90 초이며, 인증 실패 시 401 반환.
인증과 내 정보 조회	완료	auth.py, app.py, api.js, main.jsx	Bearer 토큰을 이용한 사용자 인증 및 내 정보 조회 구현. 토큰 누락·변조·만료에 대한 검증 수행.
본인 글 작성·수정·삭제	완료	app.py, auth.py, store.py, api.js, main.jsx	로그인한 사용자는 공략 게시글을 작성할 수 있으며, 수정·삭제는 작성자 본인만 가능함. 다른 작성자의 변경 요청은 403 으로 거부.
만료·로그아웃·오류 표시	완료	auth.py, errors.py, api.js, main.jsx	JWT 만료 시 401 응답과 로그인 상태 해제 구현. 로그아웃 시 브라우저 메모리의 토큰 제거. 서버 측 토큰 폐기 기능은 별도 구현하지 않음.

기능	완료 여부	관련 파일	비고 또는 제한
DB 데이터 유지	완료	store.py, app.py, data 폴더	SQLite 에 게시글과 작성자 정보를 저장하며, FastAPI 서버 종료·재실행 후에도 기존 데이터가 유지됨을 확인.

구조와 변경 내역

시스템 구조 및 요청 흐름

Mario Secret Guide 는 React 프론트엔드, FastAPI 백엔드, 데이터 저장소로 구성된 웹서비스이다. React 는 사용자 화면과 입력을 처리하고, FastAPI 는 HTTP 요청에 따른 JWT 인증, 작성자 권한 검사 및 게시글 데이터 처리를 담당한다.

1. 전체 시스템 구조

브라우저(React) → Vite 프록시 → FastAPI → 데이터 저장소

- React: 로그인 화면, 공략 게시글 목록·상세 조회, 작성·수정·삭제 화면 제공
- Vite: 프론트엔드 파일 제공 및 /api 요청을 FastAPI 8000 번 포트로 전달
- FastAPI: 로그인, JWT 발급·검증, 작성자 권한 검사, 게시글 API 처리
- 데이터 저장소: 공략 게시글과 작성자 정보의 저장 및 조회

2. 로그인 요청 흐름

React 로그인 화면 → api.js → POST /api/auth/login → app.py → users.py 에서 계정 검증 → auth.py 의 issue_token()에서 JWT 발급 → React 에 JWT 와 사용자 정보 반환

로그인에 성공하면 React 는 서버가 발급한 JWT 를 브라우저 메모리에 저장한다. 이후 인증이 필요한 API 요청에는 Authorization: Bearer <TOKEN> 헤더를 추가하여 사용자 인증을 수행한다.

3. 게시글 수정 요청 흐름

React 게시글 수정 화면 → api.js → PUT /api/posts/{id} → app.py → current_user_id() → assert_owner() → store.py → 저장 결과 반환

FastAPI 는 먼저 JWT 의 서명과 만료 시간 등을 검증하고, 토큰에서 확인한 사용자 ID 와 게시글 작성자 ID 를 비교한다. 두 값이 같으면 수정을 허용하며, 다른 작성자의 게시글이면 HTTP 403 FORBIDDEN 을 반환한다.

게시글 목록과 상세 정보는 로그인하지 않은 사용자도 조회할 수 있으며, 작성·수정·삭제는 인증된 사용자에게만 허용한다.

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
backend-fastapi/auth.py issue_token()	JWT 발급 미구현	환경변수의 비밀 키를 이용한 HS256 JWT 발급, sub·exp·iss·aud 설정	로그인 성공 시 사용자 인증 토큰 발급
backend-fastapi/auth.py current_user_id()	데모 사용자 ID 1 반환	Bearer 토큰 추출, 서명·만료·필수 클레임 검증, 실제 사용자 ID 반환	미인증 및 변조·만료 토큰의 접근 차단
backend-fastapi/auth.py assert_owner()	작성자 권한 검사 미구현	로그인 사용자 ID 와 게시글 작성자 ID 비교, 불일치 시 403 FORBIDDEN 반환	다른 사용자의 게시글 수정·삭제 방지
frontend-react/src/api.js login()	로그인 API 연결 미구현	FastAPI 로그인 API 호출 및 JWT·사용자 정보의 메모리 저장	React 로그인 기능 구현
frontend-react/src/api.js authorizationHeaders(), request()	인증 헤더 전송 및 오류 처리 미완성	Bearer 토큰 첨부, HTTP 401 발생 시 세션 초기화	보호 API 인증 및 토큰 만료 처리
frontend-react/src/api.js canEdit()	작성자 구분 없이 수정 권한 허용	로그인 사용자와 게시글 작성자 ID 비교	본인 게시글에만 수정·삭제 버튼 표시
frontend-react/src/main.jsx save() 및 화면 구성	기본 게시판 화면과 폼 처리	비로그인 글 작성 제한 및 슈퍼마리오 공략 서비스 안내 문구 변경	사용자 권한 안내와 서비스 주제 반영
frontend-react/index.html	기본 브라우저 탭 제목	Mario Secret Guide 로 제목 변경	서비스 식별성 확보
backend-fastapi/.env	환경변수 예시를 기반으로 한 로컬 설정	JWT_SECRET 무작위 키 생성 및 JWT 발급 설정 적용	토큰 서명 및 검증에 필요한 서버 설정 구성

각 변경의 작성 출처도 구분하세요. 예: 제공 코드 유지, 본인 작성, AI 초안 후 본인 수정, 외부 예제 참고. 보조 파일을 추가했다면 함께 기록하세요.

app.py, users.py, store.py 등 시작 템플릿에 제공된 기본 API 및 데이터 처리 코드를 활용하였다. JWT 발급·검증·권한 검사와 React 인증 연동 부분은 AI 가 제안한 코드 초안을 참고하여 직접 적용하였으며, 실행 과정에서 문법 검사 및 HTTP 응답을 통해 동작을 검증하였다. 화면 문구는 Mario Secret Guide 의 서비스 목적에 맞게 변경하였다.

로컬 실행 기록

실제 설치·빌드·실행 명령과 결과를 적고, 자세한 재현 방법은 저장소 README 에서 연결하세요.

기록 항목	내용
프론트엔드	React + Vite
라이브러리 설치	npm ci
개발 서버 실행	npm run dev
접속 주소	http://127.0.0.1:5173
실제 실행 결과	프론트엔드 빌드 및 4173 번 접속 성공, 백엔드 API 연결 확인, T01~T17 테스트 수행

[운영체제와 버전 / 새 폴더 설치 / 환경설정 / 프론트 빌드 / 백엔드 실행 / SQLite 파일 경로]

[4173 번 접속 및 API 연결 확인 / 서버 재시작 후 데이터 유지 / 증거 위치]

로컬 빌드 및 API 연동 검증

프론트엔드는 React 와 Vite 를 사용하여 구현하였다. 의존성 설치 후 npm run dev 명령어로 5173 번 포트의 개발 서버를 실행하였으며, npm run build 를 통해 프론트엔드 빌드를 수행하였다. 이후 npm run preview 명령어로 4173 번 포트에서 최종 빌드 결과를 실행하였다.

백엔드는 Python 가상환경에 FastAPI 관련 라이브러리를 설치하고, Uvicorn 을 사용하여 8000 번 포트에서 실행하였다. JWT 인증에 필요한 환경변수는 .env 파일에 설정하였으며, JWT 비밀 키는 외부에 노출하지 않도록 관리하였다.

최종 테스트 환경인 http://127.0.0.1:4173 에서 프론트엔드와 백엔드의 API 연동을 확인하였다. 게시글 조회, 등록, 수정, 삭제 및 JWT 인증 요청에 대한 HTTP 응답을 Chrome 개발자 도구에서 확인하였으며, T01~T17 테스트를 수행하였다.

또한 SQLite 에 저장된 게시글의 지속성을 검증하기 위해 게시글을 작성한 후 FastAPI 서버 프로세스를 종료하고 재실행하였다. 재실행 이후 동일한 게시글 ID 5 를 조회한 결과 HTTP 200 OK 가 반환되었으며, 게시글 제목, 작성자 및 생성 시각이 유지되는 것을 확인하였다.

관련 실행 및 검증 증거는 보고서의 테스트 증거 항목에 첨부하였다.

핵심 코드 설명

네 부분을 각각 설명하세요. 전체 파일을 붙이지 말고 핵심 코드 5~20 줄 정도를 발췌한 뒤 입력 → 처리 → 출력과 실패 조건을 자신의 말로 설명합니다. 성공·실패 시 HTTP 상태와 JSON code, 해당 분기를 만드는 코드도 연결해 적으세요. 줄 번호를 적는 경우 제출한 코드 기준인지 확인하세요.

3 1 로그인과 JWT 발급

파일 / 함수: backend-fastapi/auth.py - issue_token(user)

코드 발췌

```
def issue_token(user): # 로그인 성공 시 JWT 발급
    # 1. 서버 환경변수에서 비밀 키 가져오기
    secret_hex = os.getenv("JWT_SECRET", "")

    # 2. 비밀 키 형식 확인
    if not re.fullmatch(r"[0-9a-fA-F]{64}", secret_hex):
        raise RuntimeError("JWT_SECRET 설정이 올바르지 않습니다.")

    # 3. JWT 유효 시간 확인
    ttl = int(os.getenv("TOKEN_TTL", "90"))
    if ttl <= 0:
        raise RuntimeError("TOKEN_TTL 은 양수여야 합니다.")

    # 4. 발급자와 사용 대상 확인
    issuer = os.getenv("JWT_ISSUER", "")
    audience = os.getenv("JWT_AUDIENCE", "")

    if not issuer or not audience:
        raise RuntimeError("JWT 발급자 또는 사용 대상 설정이 없습니다.")

    # 5. JWT 에 담을 사용자 정보 구성
    now = int(time.time())

    payload = {
        "sub": str(user["id"]), # sub: 토큰의 주체인 사용자 id
        "exp": now + ttl, # 토큰이 만료되는 시각
        "iss": issuer, # JWT 를 발급한 주체
        "aud": audience, # JWT 를 사용할 대상
    }

    # 6. 비밀 키로 서명한 JWT 발급
    return jwt.encode(
        payload,
        bytes.fromhex(secret_hex),
```

```
        algorithm="HS256",
    )
```

동작과 응답 코드 설명

사용자가 React 로그인 화면에서 이메일과 비밀번호를 입력하면 FastAPI 의 로그인 API 로 요청이 전달된다. 서버는 users.py 의 계정 검증 기능을 통해 입력된 정보를 확인한 후, 인증에 성공한 사용자 정보를 issue_token() 함수에 전달한다.

issue_token()은 환경변수 JWT_SECRET 에서 비밀 키를 가져오고, 사용자 ID(sub), 만료 시각(exp), 발급자(iss), 사용 대상(aud)을 포함한 JWT 페이로드를 구성한다. 이후 HS256 알고리즘으로 서명된 JWT 를 발급한다.

정상 로그인 시 서버는 HTTP 200 OK 와 함께 accessToken, expiresIn, user 정보를 반환한다. 반면 비밀번호가 틀리거나 존재하지 않는 이메일로 로그인하면 HTTP 401 과 JSON code INVALID_CREDENTIALS 를 반환하며 JWT 를 발급하지 않는다.

실제 테스트 T02 에서 HTTP 200 과 JWT 발급 및 Alice 사용자 정보 반환을 확인하였으며, T03 에서 두 가지 잘못된 로그인 요청 모두 401 / INVALID_CREDENTIALS 로 거부되는 것을 확인하였다.

3 2 브라우저 토큰 저장과 전송

파일 / 함수: frontend-react/src/api.js — login(), authorizationHeaders()frontend-

코드 발췌

```
export async function login(email, password) {
  // 1. FastAPI 서버에 로그인 요청
  const data = await request("/auth/login", {
    method: "POST",
    body: JSON.stringify({ email, password }),
  });

  // 2. 서버가 반환한 JWT 와 사용자 정보 저장
  session = {
    accessToken: data.accessToken,
    user: data.user,
  };

  // 3. 로그인한 사용자 정보 반환
  return data.user;
}
function authorizationHeaders() {
```

```
  if (!session.accessToken) {
    return {};
  }

  return {
    Authorization: `Bearer ${session.accessToken}`,
  };
}
```

동작과 응답 코드 설명

[로그인 응답 처리, 저장 위치, Authorization 헤더, 로그아웃을 설명] 성공·실패 HTTP 와 JSON code 도 적으세요.

React 의 login() 함수는 사용자가 입력한 이메일과 비밀번호를 FastAPI 의 /api/auth/login 으로 전송한다. 로그인에 성공하면 서버가 반환한 JWT(accessToken)와 사용자 정보를 session 변수에 저장한다.

JWT 는 브라우저의 localStorage 가 아닌 JavaScript 메모리에 보관한다. 따라서 페이지를 새로고침하면 기존 로그인 상태가 초기화되도록 구성하였다.

authorizationHeaders() 함수는 저장된 JWT 가 존재하는 경우 HTTP 요청에 Authorization: Bearer <TOKEN> 헤더를 추가한다. 이를 통해 서버는 요청을 보낸 사용자의 신원을 검증할 수 있다.

정상 JWT 로 /api/me 를 요청하면 HTTP 200 과 현재 사용자 정보가 반환된다. 인증 토큰이 없으면 401 / TOKEN_REQUIRED, 만료된 토큰을 사용하면 401 / TOKEN_EXPIRED 가 반환된다. 프론트엔드는 HTTP 401 응답을 받으면 메모리에 저장된 인증 정보를 제거하고 로그인 상태를 해제한다.

실제 테스트 T05 에서 정상 JWT 를 이용한 사용자 정보 조회가 HTTP 200 으로 성공했으며, T10 에서 토큰 만료에 따른 401 응답과 자동 로그아웃을 확인하였다. 또한 T11 에서는 로그아웃 후 인증 없는 요청이 거부되는 것을 확인하였다.

웹서비스설계 03 계속

서버 인증과 작성자 권한

아래 설명은 실제 제출한 파일과 함수의 동작을 근거로 작성하세요.

3 3 서버 JWT 검증

파일 / 함수: backend-fastapi/auth.py / current_user_id(request)

코드 발췌

```
# 3. JWT 서명과 만료 시간, 필수 정보 검증
    try:
        payload = jwt.decode(
            token,
            bytes.fromhex(secret_hex),
            algorithms=["HS256"],
            issuer=issuer,
            audience=audience,
            options={"require": ["sub", "exp", "iss", "aud"]},
        )

    except jwt.ExpiredSignatureError:
        raise ApiError(401, "TOKEN_EXPIRED", "토큰이 만료되었습니다.")

    except jwt.InvalidTokenError:
        raise ApiError(401, "INVALID_TOKEN", "유효하지 않은 토큰입니다.")
```

동작과 응답 코드 설명

[서명·알고리즘·만료·필수 클레임 검사와 사용자 ID 전달, 실패 응답을 설명] 성공·실패 HTTP 와 JSON code 도 적으세요.

current_user_id() 함수는 클라이언트의 HTTP 요청에 포함된 JWT 를 검증하고, 정상적으로 인증된 사용자의 ID 를 반환하는 역할을 한다.

클라이언트가 API 를 요청하면 먼저 Authorization 헤더에서 Bearer 토큰을 추출한다. 토큰이 존재하지 않거나 올바른 Bearer 형식이 아니면 HTTP 401 과 JSON code TOKEN_REQUIRED 를 반환한다.

토큰이 존재하는 경우 서버 환경변수에 저장된 JWT 비밀 키를 이용하여 jwt.decode()를 실행한다. 이 과정에서 HS256 알고리즘을 지정하고 서명, 만료 시간(exp), 발급자(iss), 사용 대상(aud) 및 필수 클레임의 유효성을 검증한다.

JWT 의 유효 시간이 만료된 경우에는 ExpiredSignatureError 를 처리하여 HTTP 401 과 TOKEN_EXPIRED 를 반환한다. 서명이 변조되었거나 유효하지 않은 토큰인 경우에는 InvalidTokenError 를 처리하여 HTTP 401 과 INVALID_TOKEN 을 반환한다.

검증에 성공하면 JWT 의 sub 에서 사용자 ID 를 추출하고, 실제 존재하는 사용자 계정인지 확인한 후 해당 ID 를 반환한다. 이 ID 는 이후 게시글 작성 및 작성자 권한 검사에 사용된다.

실제 테스트에서 JWT 가 없는 요청은 401 / TOKEN_REQUIRED(T04), 정상 JWT 를 사용한 내 정보 조회는 200 / 해당 없음(T05), 서명이 변조된 JWT 는 401 / INVALID_TOKEN(T09), 만료된 JWT 는 401 / TOKEN_EXPIRED(T10)를 반환하는 것을 확인하였다.

3 4 작성자 권한 검사

파일 / 함수: backend-fastapi/auth.py / assert_owner(user_id, author_id)

코드 발췌

```
def assert_owner(user_id, author_id): # 게시글 작성자 권한 검사
    # 로그인한 사용자와 게시글 작성자가 다르면 거부
    if user_id != author_id:
        raise ApiError(
            403,
            "FORBIDDEN",
            "자신의 게시글만 수정하거나 삭제할 수 있습니다."
        )
```

동작과 응답 코드 설명

[누구와 누구를 비교하는지, 401 과 403 의 차이, 작성자 위조 방지를 설명] 성공·실패 HTTP 와 JSON code 도 적으세요.

assert_owner() 함수는 로그인한 사용자가 게시글을 수정하거나 삭제할 권한이 있는지 확인하는 역할을 한다.

사용자가 게시글 수정 또는 삭제를 요청하면 FastAPI 는 먼저 current_user_id() 함수를 통해 JWT 를 검증하고 사용자 ID 를 확인한다. 이후 데이터 저장소에서 조회한 게시글의 작성자 ID(authorId)와 인증된 사용자 ID(user_id)를 비교한다.

두 ID 가 일치하면 해당 사용자가 게시글 작성자이므로 수정 또는 삭제를 허용한다. 반면 두 ID 가 일치하지 않으면 ApiError 를 발생시켜 HTTP 403 과 JSON code FORBIDDEN 을 반환한다.

여기서 HTTP 401 과 403 은 서로 다른 의미를 가진다. HTTP 401 은 인증 토큰이 없거나 유효하지 않아 사용자를 정상적으로 인증하지 못한 경우이며, HTTP 403 은 사용자 인증은 성공했지만 해당 게시글을 변경할 권한이 없는 경우이다.

또한 게시글 작성 시 클라이언트가 임의로 authorId 또는 userId 를 지정하여 다른 사용자를 사칭하지 못하도록 입력값을 검사한다. 서버는 클라이언트가 전달한 작성자 ID 를

신뢰하지 않고, 검증된 JWT 에서 확인한 사용자 ID 를 실제 작성자로 사용한다. 허용되지 않은 작성자 ID 필드가 포함되면 HTTP 400 과 JSON code BAD_INPUT 을 반환한다.

실제 테스트 T07 과 T08 에서 Bob 이 Alice 의 게시글을 수정하거나 삭제하려는 요청에 대해 HTTP 403 / FORBIDDEN 이 반환되었으며, 기존 게시글이 변경되지 않았음을 확인하였다.

T12 에서는 Bob 이 authorId: 1 과 userId: 1 을 각각 포함하여 게시글 등록을 시도했으나 두 요청 모두 HTTP 400 / BAD_INPUT 으로 거부되었고, 위조된 게시글이 생성되지 않았다.

반면 T17 에서는 Bob 이 자신의 게시글을 정상적으로 작성·수정·삭제하여 각각 HTTP 201, 200, 204 응답을 확인하였다. 이를 통해 작성자 본인에게는 변경 권한을 부여하고 다른 사용자에게는 해당 권한을 제한하는 기능이 정상적으로 동작함을 검증하였다.

테스트 결과

T01 부터 T17 까지 수행하고, 실제 응답과 증거의 위치를 기록하세요.

실행 환경: macOS / Python 3.14.7 / Google Chrome / FastAPI(Uvicorn), React(Vite)

로컬 배포 확인: [새 폴더에서 설치한 절차 / 프론트 빌드 결과 / 백엔드 실행 명령 / 4173 번 접속 및 API 연결 증거]
테스트 일시 및 코드 버전: 2026.10.28 / 시작 템플릿 (커밋 SHA 미확인)
토큰 TTL: [일반 테스트 값 / 만료 테스트 값]
05_테스트시나리오.md 의 T01~T17 결과를 모두 기록하세요. PASS, FAIL, 미실행을 구분합니다. 예상 결과를 실제 결과로 복사하지 말고 직접 관찰한 HTTP 상태와 JSON code, 화면을 적으세요. 실제 결과 칸에는 예를 들어 403 / FORBIDDEN 처럼 기록합니다. 정상 응답은 200 / 해당 없음처럼 적습니다.

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	토큰 없이 게시글 목록 및 기존 글 상세 조회	목록·상세 모두 200, 정상 공개 조회	목록 200 / 해당 없음, 상세 200 / 해당 없음	PASS	그림 1, 그림 2
T02	Alice 계정으로 로그인	POST /api/auth/login	200 OK / 해당 없음	PASS	그림 3, 그림 4
T03	잘못된 비밀번호 및 없는 이메일로 각각 로그인	모두 401 INVALID_CREDENTIALS, JWT 미발급	401 / INVALID_CREDENTIALS	PASS	그림 5, 그림 6
T04	토큰 없이 /me 와 글 등록 요청	두 요청 모두 401 / TOKEN_REQUIRED]	내 정보 조회 401 / TOKEN_REQUIRED, 게시글 등록 401 / TOKEN_REQUIRED	PASS	그림 7, 그림 8
T05	Alice 토큰으로 글 작성 후 /me 조회	등록 201, authorId 1, /me 는 Alice	등록 201 / 해당 없음 (authorId=1), 내 정보 조회 200 / 해당 없음 (Alice, id=1)	PASS	그림 9, 그림 10
T06	Alice 토큰으로 본인 글 수정·삭제	수정 200, 삭제 204, DB 반영	수정 200 / 해당 없음, 삭제 204 / 해당 없음	PASS	그림 11, 그림 12
T07	Alice 글을 Bob 의 정상 토큰으로 수정	403 / FORBIDDEN, 원문 유지	403 / FORBIDDEN, 재조회 200 / 해당 없음, 원본 유지 확인	PASS	그림 13
T08	Alice 글을 Bob 의 정상 토큰으로 삭제	403 Forbidden	403 / FORBIDDEN, 재조회 200 / 해당 없음, 원본 유지 확인	PASS	그림 14
T09	유효 토큰의 서명 부분 첫 글자를 다른 문자로 바꿔 /me 및 변경 요청	모든 요청 401 / INVALID_TOKEN, 데이터 변경 없음	GET·POST·PUT·DELETE 모두 401 / INVALID_TOKEN, 재조회 결과 원본 유지	PASS	그림 15~18
T10	짧은 TTL 로 발급하고 실제 만료 뒤 /me 및 변경 요청	401 / TOKEN_EXPIRED, 화면 로그인 해제 및 안내	401 / TOKEN_EXPIRED, 화면 로그인 해제 및 안내	PASS	그림 19~23

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T11	로그인 후 로그아웃, 다음 /me 요청	토큰 제거 후 401 / TOKEN_REQUIRED	401 / TOKEN_REQUIRED, Authorization 헤더 미포함, 로그아웃 상태 유지	PASS	그림 24
T12	Bob 로그인 상태로 글 등록 요청에 authorId:1 또는 userId:1 추가	두 요청 모두 400 / BAD_INPUT, 위조 게시글 미생성	authorId·userId 각각 400 / BAD_INPUT, 게시글 목록 유지 확인	PASS	그림 25
T13	본인 토큰으로 빈 제목·공백 본문·길이 초과 입력을 각각 전송	모든 요청 400 / BAD_INPUT, DB 변경 없음	빈 제목·공백 본문·제목 101 자·본문 5001 자 모두 400 / BAD_INPUT, 게시글 목록 유지	PASS	그림 26
T14	없는 양의 정수 ID 의 조회·수정·삭제 및 잘못된 ID 조회	없는 글 404 / NOT_FOUND, 잘못된 ID 400 / BAD_INPUT	GET·PUT·DELETE 모두 404 / NOT_FOUND, 잘못된 ID GET 400 / BAD_INPUT, 기존 데이터 유지	PASS	그림 27
T15	글을 만든 뒤 서버 종료·재실행	서버 재시작 후 기존 글과 작성자·내용 유지	등록 201 / 해당 없음, 재시작 후 조회 200 / 해당 없음, ID 5 및 저장 데이터 유지	PASS	그림 28, 그림 29
T16	로그인 후 페이지 새로고침	공개 게시글 유지, 메모리 로그인 상태 해제	200 / 해당 없음, 기존 게시글 2 개 유지 및 로그인 상태 해제	PASS	그림 30
T17	토큰 없는 수정·삭제 요청과 Bob 계정의 정상 작성·본인 수정·삭제	무토큰 요청 401 / TOKEN_REQUIRED, Bob 본인 작업 정상 처리	무토큰 PUT·DELETE 각각 401 / TOKEN_REQUIRED, Bob POST 201 / 해당 없음, PUT 200 / 해당 없음, DELETE 204 / 해당 없음	PASS	그림 31~34

HTTP 응답 확인 기록

예상 코드와 실제 코드를 대조하고, 201·204·401 의 헤더와 본문도 확인하세요.

숫자는 Network 나 API 클라이언트에서 직접 관찰한 값을 기록합니다. 본문에 있는 숫자만으로 판단하지 않습니다. 한 항목이 여러 요청을 포함하면 행을 추가하세요.

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 조회·로그인·수정	**200**	조회·로그인·수정 모두 200 / 해당 없음	그림 1~4, 10, 11, 33
새 글 등록	**201 및 Location**	201 / 해당 없음, Location: /api/posts/6 확인	그림 32
본인 글 삭제	**204 및 빈 본문**	204 / 해당 없음 (No Content)	그림 12, 34
입력 오류·추가 작성자 필드	**400 BAD_INPUT**	400 / BAD_INPUT, 입력 오류 및 authorId·userId 위조 요청 거부	그림 25, 26
잘못된 계정	**401 INVALID_CREDENTIALS**	401 / INVALID_CREDENTIALS, JWT 미발급	그림 5, 6

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
토큰 없음·헤더 형식 오류	**401 TOKEN_REQUIRED**	401 / TOKEN_REQUIRED, 토큰 없는 요청 거부 확인	그림 7, 8, 24, 31
변조·클레임 오류	**401 INVALID_TOKEN**	401 / INVALID_TOKEN, JWT 서명 변조 요청 거부 확인	그림 15~18
정상 서명의 만료 토큰	**401 TOKEN_EXPIRED**	401 / TOKEN_EXPIRED, 만료 후 요청 거부 및 데이터 유지 확인	그림 19~23
다른 작성자의 변경	**403 FORBIDDEN**	403 / FORBIDDEN, 수정·삭제 요청 거부 및 원본 유지	그림 13, 14
없는 글	**404 NOT_FOUND**	404 / NOT_FOUND, 존재하지 않는 게시글의 조회·수정·삭제 거부	그림 27

[보호 API 401 의 WWW-Authenticate: Bearer 확인 결과와 증거]

[401 과 403 에서 화면이 각각 어떻게 달라지는지 설명]

[실패한 수정·삭제 뒤 공개 조회로 데이터 불변을 확인한 결과]

[토큰 없이 없는 글 수정 401 / 정상 토큰으로 없는 글 수정 404 의 실제 결과]

1. 보호 API 의 WWW-Authenticate: Bearer 확인

JWT 인증이 필요한 API 에 토큰 없이 접근하거나 변조·만료된 JWT 를 전송한 경우 HTTP 401 응답이 반환되었다. T09 와 T10 의 테스트에서 JSON 오류 코드와 함께 WWW-Authenticate: Bearer 응답 헤더가 포함되는 것을 확인하였다. 이를 통해 인증에 실패한 요청에 대해 서버가 Bearer 인증 방식을 명시하고 있음을 검증하였다.

2. HTTP 401 과 403 의 처리 차이

HTTP 401 은 인증 토큰이 없거나 유효하지 않은 경우 발생하며, HTTP 403 은 정상적으로 인증된 사용자에게 해당 작업의 권한이 없는 경우 발생한다.

T10 에서는 만료된 JWT 로 내 정보 조회를 요청했을 때 401 / TOKEN_EXPIRED 가 반환되었고, React 화면에서 로그인 상태가 해제되는 것을 확인하였다.

T07 과 T08 에서는 Bob 이 Alice 의 게시글을 수정·삭제하려고 시도했으나 403 / FORBIDDEN 이 반환되었다. 서버에서 해당 요청을 거부하고 원본 데이터가 유지되는 것을 확인하였다. 다만 403 발생 시 React 로그인 상태 유지 여부는 별도로 검증하지 않았다.

3. 실패한 수정·삭제 후 데이터 불변 확인

T07 과 T08 에서 Bob 이 Alice 의 게시글을 수정하거나 삭제하려는 요청은 모두 403 / FORBIDDEN 으로 거부되었다. 이후 공개 조회 API 를 통해 게시글을 다시 확인한 결과 HTTP 200 이 반환되었으며, 기존 제목·내용과 작성자 정보가 유지되었다.

또한 T09 에서는 변조된 JWT, T10 에서는 만료된 JWT 를 이용한 변경 요청이 각각 401 / INVALID_TOKEN, 401 / TOKEN_EXPIRED 로 거부되었다. 요청 실패 후 게시글을 재조회하여 원본 데이터가 변경되지 않았음을 확인하였다.

4. 토큰 유무에 따른 존재하지 않는 게시글 수정 요청

동일한 존재하지 않는 게시글 ID 1003 에 대해 JWT 유무에 따른 수정 요청 결과를 비교하였다.

Alice 의 정상 JWT 를 사용한 PUT /api/posts/1003 요청에서는 게시글이 존재하지 않아 HTTP 404 / NOT_FOUND 가 반환되었다(T14).

반면 JWT 없이 동일한 PUT /api/posts/1003 요청을 전송한 경우에는 사용자 인증 단계에서 HTTP 401 / TOKEN_REQUIRED 가 반환되었다.

이를 통해 서버가 게시글 존재 여부를 검사하기 전에 JWT 인증을 수행하며, 인증되지 않은 요청을 우선 거부하는 것을 확인하였다.

관련 증거: 그림 27, 그림 35

웹서비스설계 04 계속

테스트 증거

캡처에는 테스트 ID 와 실제 응답을 표시하세요. 토큰 원문과 서버 비밀 키는 가립니다

로그인과 토큰 전송

[증거 1: 로그인 성공과 토큰이 첨부된 요청. 토큰 원문은 가리세요.]

[화면 캡처 또는 요청·응답 삽입]

Alice 계정으로 로그인한 결과 POST /api/auth/login 요청에 대해 HTTP 200 OK 가 반환되었으며, JWT(accessToken), 유효 시간(expiresIn), 사용자 정보가 정상적으로 발급되었다.

발급된 JWT 는 React 의 메모리에 저장되며, 인증이 필요한 API 요청 시 Authorization: Bearer 형식으로 서버에 전달된다. 정상 JWT 를 사용한 GET /api/me 요청에서 HTTP 200 OK 와 Alice 사용자 정보가 반환되는 것을 확인하였다.

관련 증거: 그림 3, 4, 10

다른 작성자의 수정과 삭제

[증거 2: Bob 이 Alice 의 글 수정·삭제를 시도한 서버 응답]

[화면 캡처 또는 요청·응답 삽입]

Bob 의 정상 JWT 를 이용하여 Alice 가 작성한 게시글 ID 3 에 수정(PUT) 및 삭제(DELETE) 요청을 전송하였다.

두 요청 모두 HTTP 403 / FORBIDDEN 으로 거부되었으며, 이후 공개 조회 API 를 통해 기존 게시글의 제목, 내용 및 작성자 정보가 변경되지 않았음을 확인하였다.

관련 증거: 그림 13, 14

토큰 변조와 만료

[증거 3: 토큰 변조·만료 시 서버 응답과 화면 처리]

[화면 캡처 또는 요청·응답 삽입]

정상적으로 발급된 JWT 의 서명 부분 첫 글자를 변경한 후 GET, POST, PUT, DELETE 요청을 전송한 결과, 모든 요청에서 HTTP 401 / INVALID_TOKEN 이 반환되었다. 변경 요청이 거부된 이후에도 기존 게시글이 유지되는 것을 확인하였다.

JWT 만료 검증을 위해 토큰 유효 시간을 3 초로 설정하였다. 실제 만료 후 API 요청에서 HTTP 401 / TOKEN_EXPIRED 가 반환되었으며, React 화면의 로그인 상태가 해제되는 것을 확인하였다.

만료된 JWT 를 사용한 게시글 등록·수정·삭제 요청 역시 거부되었으며, 기존 데이터가 유지되었다.

관련 증거: 그림 15~23

자동 검사와 로컬 배포 증거

[자동 검사 결과를 첨부했다면 실행 명령과 의미를 설명]

[빌드 성공 로그 / 4173 번 API 연결 / 재시작 유지의 화면 또는 파일 경로]

프론트엔드에서 npm run build 및 npm run preview 를 사용하여 빌드 결과를 4173 번 포트에서 실행하였다. FastAPI 백엔드는 Uvicorn 을 이용하여 8000 번 포트에서 실행하였다.

최종 검증 환경인 http://127.0.0.1:4173 에서 React 와 FastAPI 간 API 연결을 확인하고, Chrome 개발자 도구 및 직접 API 요청을 통해 T01~T17 시나리오를 검증하였다.

또한 게시글 ID 5 를 생성한 후 FastAPI 서버 프로세스를 종료하고 재실행하였다. 재실행 후 GET /api/posts/5 요청에서 HTTP 200 OK 가 반환되었으며, 기존 게시글의 제목과 작성자 및 생성 시각이 유지되는 것을 확인하였다.

별도의 자동 테스트 실행 결과는 현재 보고서에 포함하지 않았으며, 위 검증은 수동 API 및 화면 테스트 결과를 기준으로 작성하였다.

관련 증거: 그림 28~30 및 T01~T17 테스트 결과표

웹서비스설계 05

AI 활용과 참고 자료

사용 범위, 실제 반영 위치와 직접 확인한 내용을 구분해 기록하세요.

AI 사용 여부: 사용
사용하지 않았다면 미사용이라고 명시하고 참고한 수업 자료·공식 문서의 출처를 작성하세요. 사용했다면 전체 사용 범위를 요약하고 대표 사례 2~3 건을 작성하세요. 사용이 1 건이면 그 1 건만 적습니다. 대화 전체 제출은 필요하지 않습니다.

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
ChatGPT	JWT 비밀 키 생성 및 설정 방법 안내	backend-fastapi/.env	제안 명령어 참고 후 직접 실행 및 검증
ChatGPT	로그인 요청 및 JWT 저장 코드 작성 보조	frontend-react/src/api.js / login()	AI 제안 코드 검토 후 수정 반영
ChatGPT	JWT 를 Authorization 헤더에 첨부하는 기능 구현	frontend-react/src/api.js / authorizationHeaders()	AI 제안 코드 참고 후 반영
ChatGPT	HTTP 401 발생 시 로그인 상태 초기화	frontend-react/src/api.js / request()	AI 제안 코드 검토 후 반영
ChatGPT	게시글 작성자별 수정·삭제 버튼 표시	frontend-react/src/api.js / canEdit()	AI 제안 코드 참고 후 반영

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
ChatGPT	비로그인 사용자의 게시글 작성 제한	frontend-react/src/main.jsx / save()	AI 제안 코드를 기존 함수에 추가
ChatGPT	API 오류 및 권한 검증 테스트 코드 작성 보조	Chrome 개발자 도구 Console, T01~T17 테스트 결과	테스트 코드 참고 후 직접 실행하고 실제 응답 기록
ChatGPT	프론트엔드·백엔드 구조와 JWT 인증 개념 학습	보고서 02. 구조와 변경 내역, 03. 핵심 코드 설명	개념을 이해한 후 실제 구현 구조와 비교하여 설명 작성

대표 사례 1. JWT 비밀 키 생성 및 환경변수 설정

도구 / 날짜: ChatGPT / 2026.10.28

요청한 내용: FastAPI 에서 JWT 발급에 필요한 비밀 키를 안전하게 생성하고, 환경변수 파일에 설정하는 방법을 문의하였다.

AI 가 제안한 내용: Python 의 secrets 모듈을 사용하여 무작위 비밀 키를 생성하고, .env 파일의 JWT_SECRET 항목에 저장하는 명령어를 제안하였다. 또한 생성된 키가 64 자리 16 진수 형식인지 정규표현식으로 검사하는 방법을 안내하였다.

실제 반영 위치: backend-fastapi/.env 의 JWT_SECRET 환경변수

내가 확인하고 수정한 내용: VS Code 터미널에서 비밀 키 생성 명령어를 직접 실행하였다. 이후 설정 검증 명령어를 실행하여 정상이라는 출력 결과를 확인하였다. 생성된 비밀 키의 실제 값은 코드나 보고서에 공개하지 않았다.

검증 방법과 결과: JWT 발급 기능 구현 후 Alice 계정으로 로그인을 수행한 결과 HTTP 200 OK 와 JWT 발급을 확인하였다(T02). 또한 서명을 변조한 JWT 가 401 / INVALID_TOKEN 으로 거부되는 것을 확인하였다(T09).

대표 사례 2. React 와 FastAPI 간 JWT 인증 연동

도구 / 날짜: ChatGPT / 2026.10.28

요청한 내용: FastAPI 에서 발급한 JWT 를 React 에 저장하고, 로그인 이후 API 요청에 자동으로 첨부하는 방법을 문의하였다.

AI 가 제안한 내용: login() 함수에서 서버의 로그인 API 를 호출한 후 반환된 accessToken 과 사용자 정보를 메모리에 저장하고, authorizationHeaders() 함수에서 JWT 를 Authorization: Bearer <TOKEN> 형식으로 요청 헤더에 추가하도록 제안하였다.

실제 반영 위치: frontend-react/src/api.js 의 login(), authorizationHeaders()

내가 확인하고 수정한 내용: 기존 시작 템플릿의 로그인 및 인증 헤더 처리 함수를 수정하였다. JWT 를 브라우저 메모리에 저장하고 인증이 필요한 API 를 요청할 때만 Bearer 헤더를 추가하도록 구성하였다.

검증 방법과 결과: Alice 로그인 요청에서 HTTP 200 OK 와 JWT 발급을 확인하였다(T02). 로그인 후 내 정보 조회 요청에서 Authorization 헤더가 전송되고 HTTP 200 OK 와 Alice 사용자 정보가 반환되는 것을 확인하였다(T05). 또한 로그아웃 후 요청에서 401 / TOKEN_REQUIRED 가 반환되는 것을 확인하였다(T11).

대표 사례 3. JWT 만료 처리 및 작성자 권한에 따른 화면 제어

도구 / 날짜: ChatGPT / 2026.10.28

요청한 내용: JWT 가 만료되었을 때 React 화면의 로그인 상태를 해제하고, 게시글 작성자에게만 수정·삭제 버튼을 표시하는 방법을 문의하였다.

AI 가 제안한 내용: request() 함수에서 HTTP 401 응답을 받으면 clearSession()을 호출하여 메모리의 인증 정보를 제거하도록 제안하였다. 또한 canEdit() 함수에서 현재 로그인 사용자 ID 와 게시글 작성자 ID 를 비교하여 수정·삭제 버튼의 표시 여부를 결정하도록 안내하였다.

실제 반영 위치: frontend-react/src/api.js 의 request(), canEdit()

내가 확인하고 수정한 내용: HTTP 401 응답 발생 시 세션이 초기화되도록 수정하였다. 작성자 권한 검사에서는 현재 로그인한 사용자와 게시글의 작성자 ID 가 동일한 경우에만 수정·삭제 버튼을 표시하도록 변경하였다.

검증 방법과 결과: JWT 유효 시간을 3 초로 설정한 후 토큰 만료 상태에서 내 정보 조회를 실행하였다. 서버가 401 / TOKEN_EXPIRED 를 반환하자 React 화면이 로그아웃 상태로 전환되는 것을 확인하였다(T10).

또한 Alice 와 Bob 계정으로 각각 로그인하여 작성자에 따라 수정·삭제 버튼이 다르게 표시되는 것을 확인하였다. 백엔드 권한 검사에서도 Bob 이 Alice 의 게시글을 변경하려는

요청이 403 / FORBIDDEN 으로 거부되고 기존 데이터가 유지되는 것을 확인하였다(T07, T08).

AI 활용 결과 및 직접 검증 내용

AI 활용 과정에서 프론트엔드의 화면 제어와 백엔드의 권한 검사가 서로 다른 역할을 수행한다는 점을 이해하였다.

예를 들어 React 에서 수정·삭제 버튼을 숨기더라도 사용자가 API 를 직접 호출할 수 있기 때문에 FastAPI 에서도 JWT 인증과 작성자 권한 검사를 반드시 수행해야 한다는 점을 학습하였다.

또한 게시글 작성 시 클라이언트가 전달한 authorId 나 userId 를 그대로 신뢰해서는 안 되며, 서버가 검증된 JWT 에서 사용자 ID 를 추출하여 작성자를 결정해야 한다는 점을 이해하였다.

이러한 내용을 실제 코드에 적용한 후 Chrome 개발자 도구에서 API 요청을 직접 실행하여 HTTP 상태, JSON 오류 코드, 게시글 데이터 유지 여부를 검증하였다. 특히 JWT 누락·변조·만료, 작성자 권한 오류 및 잘못된 입력값에 대한 서버 응답을 확인하였다.

AI 가 제공한 설명과 코드 초안을 그대로 최종 결과로 간주하지 않고, 직접 실행한 테스트 결과를 기준으로 기능의 정상 작동 여부를 판단하였다.

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
웹서비스설계 02 주차 - 다양한 웹서비스: 주소 하나가 화면이 되기까지 (수업 제공 PDF)	frontend-react/src/api.js, backend-fastapi/app.py	HTTP GET·POST·PUT·DELETE 요청과 응답 구조, 상태 코드 및 JSON 데이터 교환 개념을 참고하여 React 와 FastAPI 간 API 통신을 구현함. **참고: 02 주차 강의자료 18~24, 49~50 쪽**
웹서비스설계 03 주차 - 게시판 서비스 설계와 구현 (수업 제공 PDF)	backend-fastapi/app.py, store.py, frontend-react/src/main.jsx	REST API 설계, 게시글 CRUD, 입력값 검증, DB 저장 및 HTTP 200·201·204·400·404 응답 처리 개념을 참고함. 강의의 Vue·Express 예제를 React·FastAPI 구조에 맞게 적용함. **참고: 03 주차 강의자료 20~27, 35~50 쪽**

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
웹서비스설계 06 주차 - JWT 로그인과 인증 (수업 제공 PDF)	backend-fastapi/auth.py, frontend-react/src/api.js	JWT 발급과 서명 검증, sub·exp·iss·aud 클레임, Authorization Bearer 헤더 전송, 브라우저 메모리의 토큰 저장 및 만료·로그아웃 처리에 관한 개념을 참고함. **참고: 06 주차 강의자료 28~41, 47~49 쪽**
웹서비스설계 07 주차 - JWT 접근 제어와 Swagger API 검증 (수업 제공 PDF)	backend-fastapi/auth.py 의 assert_owner(), app.py, frontend-react/src/api.js 의 canEdit()	인증과 인가의 구분, JWT 에서 검증한 사용자 ID 를 이용한 작성자 권한 검사, 작성자 ID 위조 방지, HTTP 401·403 응답 처리 및 원본 데이터 유지 검증 방법을 참고함. **참고: 07 주차 강의자료 40~41, 59~69, 104 쪽**

웹서비스설계 06

이해도 확인과 회고

각 질문에 본인 코드의 동작을 근거로 답하세요.

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

클라이언트가 전송한 authorId 를 그대로 신뢰하면 다른 사용자의 이름으로 게시글을 작성하는 작성자 위조 문제가 발생할 수 있다. 예를 들어 Bob 의 사용자 ID 는 2 이지만 게시글 등록 요청에 authorId: 1 을 포함하면 Alice 가 작성한 것처럼 게시글을 등록할 수 있게 된다.

이를 방지하기 위해 app.py 의 게시글 등록 API 에서는 fields() 함수를 이용하여 title 과 content 만 입력값으로 허용한다. authorId 나 userId 등 허용하지 않은 필드가 추가되면 HTTP 400 / BAD_INPUT 을 반환하도록 구성하였다.

또한 게시글의 작성자 ID 는 클라이언트가 전달한 값이 아니라 current_user_id()를 통해 검증된 JWT 에서 추출한 사용자 ID 를 사용한다.

T12 에서 Bob 의 정상 토큰으로 authorId: 1 과 userId: 1 을 각각 포함하여 등록을 시도한 결과, 두 요청 모두 HTTP 400 / BAD_INPUT 으로 거부되었으며 새로운 게시글이 생성되지 않았음을 확인하였다.

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

React 에서 수정·삭제 버튼을 숨기는 것은 사용자가 자신의 권한에 맞는 기능을 쉽게 사용할 수 있도록 하는 화면 제어일 뿐, 실제 보안 기능을 완전히 보장하지는 않는다.

사용자는 Chrome 개발자 도구나 API 요청 도구를 사용하여 React 화면을 거치지 않고 FastAPI 서버에 직접 수정·삭제 요청을 전송할 수 있기 때문이다.

따라서 서버에서도 독립적으로 권한을 검사해야 한다. 본 과제에서는 auth.py 의 assert_owner(user_id, author_id) 함수가 JWT 에서 확인한 사용자 ID 와 데이터 저장소에 기록된 게시글 작성자 ID 를 비교한다.

두 값이 일치하면 수정·삭제를 허용하지만, 일치하지 않으면 HTTP 403 / FORBIDDEN 을 반환한다.

실제 T07 과 T08 에서 Bob 의 정상 JWT 로 Alice 의 게시글을 수정·삭제하려는 요청을 전송하였으나 모두 403 / FORBIDDEN 으로 거부되었다. 이후 게시글을 재조회하여 원본 데이터가 유지되는 것도 확인하였다.

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

JWT 의 decode 는 토큰에 포함된 Header 와 Payload 를 해석하여 내부 정보를 읽는 것을 의미한다. JWT 의 Header 와 Payload 는 Base64url 로 인코딩되어 있으므로 비밀 키 없이도 내용을 확인할 수 있다.

그러나 토큰의 내용을 읽을 수 있다는 것과 그 내용이 신뢰할 수 있다는 것은 서로 다르다. Verify 는 서버의 비밀 키와 검증 조건을 이용하여 토큰의 서명과 유효성을 확인하는 과정이다.

본 과제에서는 Python 의 PyJWT 라이브러리를 사용하였다. auth.py 의 current_user_id() 함수에서 jwt.decode()를 호출할 때 HS256 알고리즘, 서버 비밀 키, 발급자(iss), 사용 대상(aud) 및 필수 클레임을 검증하도록 구성하였다. PyJWT 의 jwt.decode()는 검증 기능이 활성화된 기본 설정에서 단순한 데이터 해석뿐 아니라 서명 및 만료 검증도 수행한다.

정상 토큰은 사용자 ID 를 반환하여 후속 API 처리를 허용하지만, 서명이 변조된 토큰은 401 / INVALID_TOKEN, 만료된 토큰은 401 / TOKEN_EXPIRED 로 거부한다.

T09 에서 JWT 서명을 변경하여 GET·POST·PUT·DELETE 요청을 전송한 결과 모두 401 / INVALID_TOKEN 이 반환되었다. T10 에서는 정상 발급된 JWT 의 유효 시간이 경과한 뒤 동일한 유형의 요청을 전송하여 401 / TOKEN_EXPIRED 가 반환되는 것을 확인하였다.

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

로그아웃은 React 에서 메모리에 저장된 JWT 와 사용자 정보를 제거하여 현재 브라우저 화면의 로그인 상태를 해제하는 동작이다. T11 에서 로그아웃 후 내 정보 조회를 요청한 결과 401 / TOKEN_REQUIRED 가 반환되었다.

토큰 만료는 JWT 에 저장된 만료 시각(exp)이 지나 서버에서 더 이상 유효한 토큰으로 인정하지 않는 상태이다. api.js 의 request() 함수는 HTTP 401 응답을 받으면 clearSession()을 호출하여 브라우저의 인증 정보를 제거하도록 구성하였다. T10 에서는 만료된 JWT 에 대해 401 / TOKEN_EXPIRED 가 반환되고 화면이 로그아웃 상태로 전환되는 것을 확인하였다.

페이지 새로고침은 React 애플리케이션을 다시 실행하는 동작이다. 본 과제에서는 JWT 를 localStorage 가 아닌 JavaScript 메모리에 저장하므로 새로고침하면 로그인 상태가 초기화된다. T16 에서 새로고침 후 로그아웃 상태로 변경되었지만 기존 게시글은 유지되는 것을 확인하였다.

다만 로그아웃이나 새로고침을 수행하더라도 이미 다른 곳에 복사된 JWT 가 서버에서 즉시 무효화되는 것은 아니다. 서명이 정상이고 만료 시간이 지나지 않았다면 복사된 토큰은 계속 사용될 수 있다.

이를 방지하려면 서버 측 토큰 폐기 목록이나 별도의 토큰 철회 정책이 필요하지만, 해당 기능은 이번 과제의 구현 범위에 포함하지 않았다

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

발생한 문제

FastAPI 백엔드 서버를 실행하는 과정에서 기존에 실행 중이던 다른 프로젝트의 서버가 8000 번 포트를 사용하고 있어 포트 충돌이 발생하였다.

원인 확인

Uvicorn 서버를 실행할 때 동일한 포트를 여러 프로세스가 동시에 사용하려고 하면서 문제가 발생한 것으로 파악하였다. 이를 통해 프로그램 코드에 문법 오류가 없더라도 실행 환경이나 포트 점유 상태 때문에 서버가 정상적으로 실행되지 않을 수 있다는 점을 알게 되었다.

수정 과정

기존에 실행 중이던 서버 프로세스를 확인하고 종료하여 8000 번 포트를 사용할 수 있도록 하였다. 이후 Python 가상환경에서 Uvicorn 실행 명령어를 다시 입력하여 FastAPI 서버를 실행하였다.

재검증 결과

서버 재실행 후 Uvicorn 의 정상 시작 로그를 확인하였고, GET /api/health 요청에서 HTTP 200 OK 와 서버 상태 정보를 반환받았다. 이후 React 에서 FastAPI API 를 호출하여 프론트엔드와 백엔드가 정상적으로 연결되는 것을 확인하였다.

현재 남은 문제와 다음에 개선할 점: [없으면 없음으로 표시]

T01~T17 테스트를 수행한 결과, 이번 과제의 핵심 요구사항인 JWT 발급·검증, 작성자 권한 검사, 게시글 CRUD 및 데이터 유지 기능에서 현재 확인된 미해결 기능 오류는 없다.

다만 향후 개선할 부분은 다음과 같다.

첫째, 현재 JWT 는 브라우저 메모리에 저장되며 로그아웃 시 클라이언트의 인증 정보만 제거된다. 향후에는 Refresh Token 및 서버 측 토큰 철회 기능을 추가하여 인증 유지와 보안 수준을 개선할 수 있다.

둘째, 이번 과제에서는 Chrome 개발자 도구와 직접 작성한 API 요청을 통해 기능을 수동 검증하였다. 향후에는 pytest 등 자동 테스트 도구를 활용하여 인증 실패, 작성자 권한 및 입력값 검증을 반복적으로 검사할 수 있도록 개선하고자 한다.

셋째, 현재 Mario Secret Guide 는 슈퍼마리오 비밀 맵 공략을 공유하는 기본 게시판 구조로 구현되어 있다. 향후에는 월드별 공략 분류, 검색 기능 및 이미지 첨부 기능을 추가하여 게임 공략 서비스로서의 활용도를 높이고자 한다.

제출 확인: [ ] 실제 코드와 설명 일치 [ ] 테스트 직접 수행 [ ] AI 활용 및 출처 기록 [ ] 비밀 키와 토큰 제거

GitHub 확인: [ ] 루트 README 작성 [ ] 보고서 PDF 와 증거 업로드 [ ] README 문서 링크 확인 [ ] 교수자 접근 가능 [ ] 저장소 URL 과 커밋 SHA 제출